Nguyễn Dương Bảo Trân

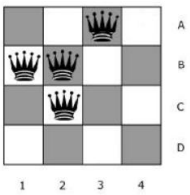
MSSV: 3123410386

BÀI TẬP CSP VÀ GAME TRONG AI

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

Đề cho trạng thái ban đầu của 4 quân hậu và giả mã Hill Climbing. Thuật toán sẽ tìm **trạng thái lân cận có số xung đột giữa các quân hậu nhỏ nhất**; nếu trạng thái tốt nhất không giảm số xung đột thì thuật toán dừng.

Từ hình ở trang 1, trạng thái ban đầu là: $A3, B1, B2, C2$.

a/ Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Trạng thái ban đầu là $A3, B1, B2, C2$, có 4 cặp quân hậu xung đột. Sau một vòng lặp, trạng thái lân cận tốt nhất đạt được bằng cách di chuyển quân hậu từ $B2$ đến $D4$. Trạng thái mới là:

$$A3, B1, C2, D4$$

Số xung đột giảm từ 4 xuống còn 1.

b/ Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☐ B) Cực trị cục bộ (local optimum)

☐ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

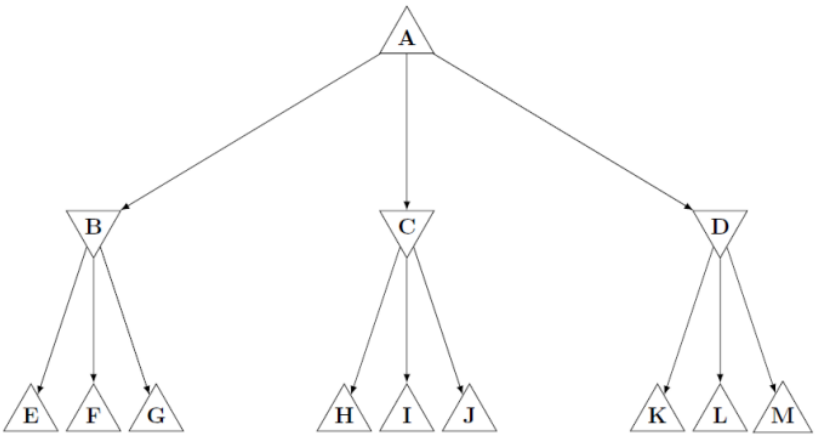
Có, thuật toán dừng. Sau bước đầu tiên, trạng thái đạt được là $A3, B1, C2, D4$ với 1 xung đột. Trạng thái lân cận tốt nhất tiếp theo cũng có 1 xung đột, nên theo điều kiện của thuật toán, vì giá trị mới không tốt hơn giá trị hiện tại nên thuật toán dừng.

Trạng thái cuối là một shoulder, vì tồn tại các trạng thái lân cận có cùng giá trị đánh giá và nếu cho phép di chuyển ngang qua các trạng thái này thì sau đó vẫn có thể tìm được trạng thái tốt hơn.

Chọn D) Vai bằng (shoulder)

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a/ Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☐ D → C → B

A là MAX nên theo góc nhìn của A, giá trị càng lớn càng tốt. Ta có:

$$f(B) = 4, f(C) = 14, f(D) = 19$$

Sắp xếp từ lớn đến nhỏ nên thứ tự là $D \rightarrow C \rightarrow B$

b/ Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Bước 1: duyệt D

D là nút **MIN.** Các con: $K = 23, L = 20, M = 26$

Ban đầu tại root A: $\alpha = -\infty, \beta = +\infty$

Xét K = 23: Tại D: $\beta = \min(+\infty, 23) = 23 \rightarrow$ Chưa thể prune.

Xét L = 20: $\beta = \min(23, 20) = 20$

Xét M = 26: $\beta = \min(20, 26) = 20$

Vậy D trả về: 20

A là MAX nên cập nhật: $\alpha = \max(-\infty, 20) = 20$

Sau khi xét D: $\alpha_A = 20$

Bước 2: duyệt C

C là MIN. Hiện tại root A đã có: $\alpha = 20$

Các con C là: $H = 17, I = 16, J = 15$

Xét H = 17:

Tại C: $\beta = \min(+\infty, 17) = 17$

Kiểm tra điều kiện cắt tỉa: $\beta \leq \alpha => 17 \leq 20$ => Đúng.

Do đó các nút còn lại của C không cần xét: I, J bị prune.

C trả về: 17

A vẫn giữ: $\alpha = \max(20, 17) = 20$

Bước 3: duyệt B

B là MIN. Root vẫn có: $\alpha = 20$

Các con: $E = 5, F = 7, G = 8$

Xét E = 5:

Tại B: $\beta = \min(+\infty, 5) = 5$

Kiểm tra điều kiện cắt tỉa: $\beta \leq \alpha => 5 \leq 20$ => Đúng.

Do đó không cần xét tiếp: F, G bị prune.

Các nút bị cắt tỉa: I, J, F, G

Giá trị cuối cùng của A

Ta có: $D = 20, C = 17, B = 5$

A là MAX: $A = \max(20, 17, 5)$ => $A = 20$

Vì vậy nếu chỉ xét đến độ sâu này, MAX chọn: D

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

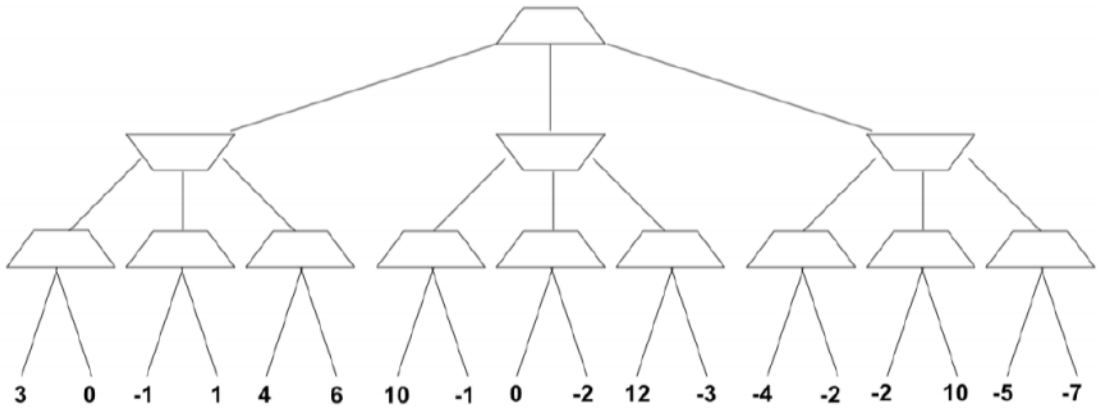
☐ Có

☐ Không

Alpha-Beta Pruning chuẩn cho cùng kết quả với Minimax khi hai thuật toán tìm kiếm trên cùng một cây và cùng độ sâu. Tuy nhiên, trong phương pháp trên, tìm kiếm bị giới hạn độ sâu và sử dụng hàm đánh giá $f(s)$ tại các nút chưa phải trạng thái kết thúc. Các giá trị đánh giá chỉ là ước lượng nên có thể khác giá trị Minimax thật khi mở rộng toàn bộ cây. Vì vậy iterative deepening kết hợp pruning ở một độ sâu giới hạn không luôn đảm bảo chọn cùng nước đi với Minimax trên toàn bộ cây.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Ký hiệu nút gốc là A, ba nút con lần lượt là B,C, D. Ba nút này là nút **MIN**, còn các nút ở tầng kế tiếp là **MAX**.

Các giá trị lá từ trái sang phải như sau:

$$B_1: (3,0), B_2: (-1,1), B_3: (4,6)$$

$$C_1: (10,-1), C_2: (0,-2), C_3: (12,-3)$$

$$D_1: (-4,-22), D_2: (-2,10), D_3: (-5,-7)$$

Bước 1: Xét nút gốc A

Ban đầu tại nút MAX A: $\alpha = -\infty, \beta = +\infty$

Duyệt nhánh bên trái trước, tức là xét nút B.

Bước 2: Xét nút B

B là nút MIN, ban đầu: $\alpha = -\infty, \beta = +\infty$

Xét B1, đây là nút MAX có hai lá là 3 và 0: $B_1 = \max(3,0) = 3$

Khi đó tại B: $\beta = \min(+\infty, 3) = 3$

Tiếp theo xét B2, có hai lá là -1 và 1: $B_2 = \max(-1,1) = 1$

Cập nhật tại B: $\beta = \min(3,1) = 1$

Tiếp theo xét B3 nhận: $\alpha = -\infty, \beta = 1$

Xét lá đầu tiên có giá trị 4: $\alpha = \max(-\infty, 4) = 4$

Lúc này $\alpha \geq \beta$ $(do\ 4 \geq 1)$ nên xảy ra cắt tỉa. Lá còn lại có giá trị 6 không cần được duyệt.

Như vậy $B = \min(3,1, \geq 4) = 1$

Nút B trả về giá trị 1.

Quay lại A: $\alpha_A = \max(-\infty, 1) = 1$

Bước 3: Xét nút C

C là nút MIN và nhận: $\alpha = 1, \beta = +\infty$

Xét C1, có hai lá 10 và -1: $C_1 = \max(10, -1) = 10$

Cập nhật: $\beta_C = \min(+\infty, 10) = 10$

Tiếp theo xét C2, có hai lá 0 và -2: $C_2 = \max(0, -2) = 0$

Cập nhật tại C: $\beta_C = \min(10, 0) = 0$

Ta có: $\beta_C \leq \alpha$ $(do\ 0 \leq 1)$ nên xảy ra cắt tỉa tại nút C. Vì vậy toàn bộ nhánh C3 gồm hai lá (12, -3) không cần được duyệt.

Nút C trả về: 0

Quay lại A: $\alpha_A = \max(1,0) = 1$

Bước 4: Xét nút D

D là nút MIN và nhận: $\alpha = 1, \beta = +\infty$

Xét D1, có hai lá -4 và -22: $D_1 = \max(-4, -22) = -4$

Cập nhật tại D: $\beta_D = \min(+\infty, -4) = -4$

Khi đó: $\beta_D \leq \alpha$ $(do -4 \leq 1)$ nên xảy ra cắt tỉa. Do đó hai nhánh còn lại là D2 và D3 không cần xét. Các lá bị cắt gồm: -2, 10, -5, -7.

Nút D trả về: -4

Bước 5: Tính giá trị tại nút gốc

Ta có: $B = 1, C = 0, D = -4$

Nút A là MAX nên: $A = \max(1, 0, -4) = 1$

Vì giá trị lớn nhất đến từ nhánh B nên nước đi tối ưu của MAX là chọn nhánh B.

Các nhánh bị cắt tỉa: các lá không được thăm do Alpha-Beta Pruning là:

$$6, 12, -3, -2, 10, -5, -7$$

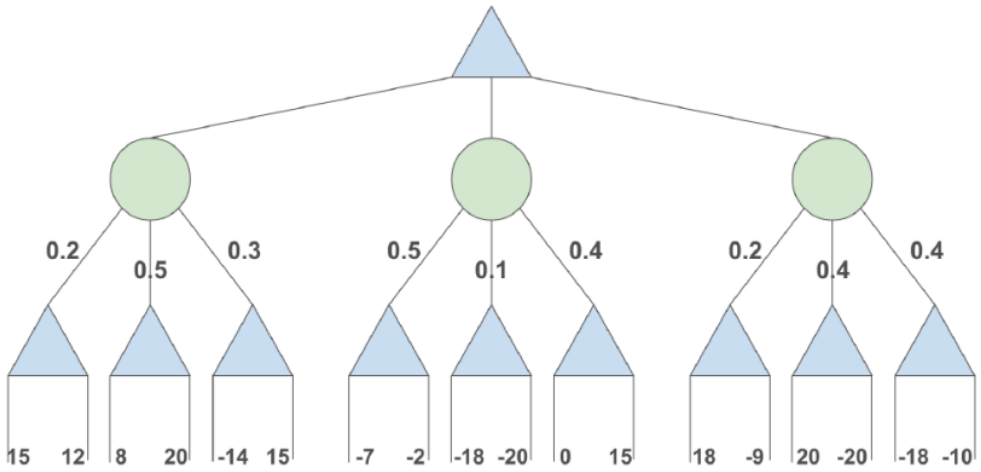
Cụ thể: tại B3 cắt lá 6; tại C cắt toàn bộ C3 gồm 12, -3; tại D cắt toàn bộ D2, D3 gồm -2, 10, -5, -7.

Giá trị Minimax tại nút gốc là: 1 và nước đi tối ưu của MAX là: chọn nhánh B.

Alpha-Beta Pruning giúp loại bỏ các nhánh không thể ảnh hưởng đến quyết định cuối cùng nhưng vẫn giữ nguyên kết quả Minimax.

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a/ Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Chance node C1

Ba nút MAX bên dưới có các cặp lá: (15,12), (8,20), (-14,15)

Vì là nút MAX nên M1 = max(15,12) = 15, M2 = max(8,20) = 20, M3 = max(-14,15) = 15

Xác suất của ba nhánh lần lượt là: 0.2, 0.5, 0.3

Do đó giá trị kỳ vọng $C1 = 0.2(15) + 0.5(20) + 0.3(15) = 3 + 10 + 4.5 = 17.5$

Chance node C2

Ba nút MAX có: (-7,-2), (-18,-20), (0,15)

Giá trị tại các nút MAX: M4 = -2, M5 = -18, M6 = 15

Xác suất: 0.5, 0.1, 0.4

Do đó $C2 = 0.5(2) + 0.1(-18) + 0.4(15) = -1 - 1.8 + 6 = 3.2$

Chance node C3

Ba nút MAX có: (18,-9), (20,-20), (-18,-10)

Giá trị tại các nút MAX: M7 = 18, M8 = 20, M9 = -10

Xác suất: 0.2, 0.4, 0.4

Do đó $C3 = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$

Giá trị nút gốc

Nút gốc là MAX nên chọn giá trị lớn nhất trong ba chance node:

$$Root = \max(17.5, 3.2, 7.6) = 17.5$$

Vì vậy MAX chọn nhánh bên trái C1

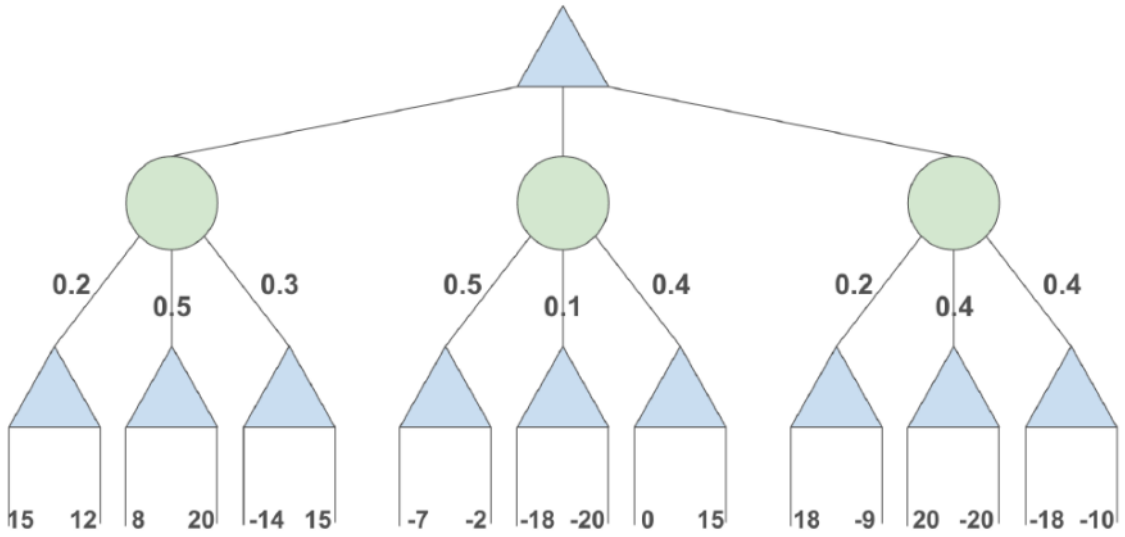
b/ Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích

Khi chưa biết giới hạn của giá trị lá, không thể cắt tỉa an toàn bất kỳ nhánh nào trong cây Expectimax. Nguyên nhân là giá trị của chance node được tính theo kỳ vọng:

$$V(C) = \sum_i p_i V_i$$

Một nhánh chưa được duyệt có thể nhận giá trị rất lớn hoặc rất nhỏ và làm thay đổi kết quả kỳ vọng. Vì vậy muốn đảm bảo kết quả chính xác, cần duyệt tất cả các nhánh.

c/ Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Khi biết mọi lá đều nằm trong đoạn [-20,20], ta có thể dùng giới hạn trên của các nhánh chưa duyệt để cắt tỉa.

Cây con trái C_1 được tính đầy đủ và có giá trị 17.5, do đó tại root ta có giá trị tốt nhất hiện tại là 17.5.

Với C_2, sau khi duyệt nút MAX đầu tiên ta được $M_4 = -2$. Phần đã biết đóng góp $0.5(-2) = -1$. Hai nhánh còn lại có tổng xác suất 0.5, và giá trị lớn nhất của chúng là 20 nên $C_2^{max} = -1 + 0.5(20) = 9$

Vì $9 < 17.5$, C_2 không thể trở thành lựa chọn tốt nhất của root. Do đó có thể cắt hai nhánh còn lại, gồm các lá -18, -20, 0, 15.

VớiC_3, nút MAX đầu tiên cho giá trị 18. Nút MAX thứ hai gặp ngay lá 20. Vì 20 là giới hạn trên của tất cả các lá, giá trị MAX chắc chắn bằng 20 nên lá -20 còn lại có thể cắt. Nhánh cuối vẫn phải xét và có giá trị $\max(-18, -10) = -10$. Vì vậy:

$$C_3 = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$$

Cuối cùng: $Root = \max(17.5, 3.2, 7.6) = 17.5$

Nút gốc chọn **nhánh bên trái**.

Các lá bị cắt là: -18, -20, 0, 15 trong cây C2 và lá -20 trong cây C3.

Bài 5. Câu hỏi thảo luận về CSP

a/ Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking	AC-3
Phạm vi lan truyền	Chỉ các biến chưa gán nối trực tiếp với biến vừa gán	Lan truyền qua nhiều cung trong mạng CSP
Thời điểm	Sau mỗi phép gán	Có thể trước hoặc trong quá trình tìm kiếm
Mức thu hẹp miền	Ít hơn	Mạnh hơn
Khả năng phát hiện mâu thuẫn	Phát hiện khi miền biến lân cận bị rỗng	Có thể phát hiện mâu thuẫn sâu hơn
Chi phí tính toán	Thấp hơn	Cao hơn

Forward Checking chỉ kiểm tra các biến chưa được gán có ràng buộc trực tiếp với biến vừa được gán và loại các giá trị xung đột khỏi miền của chúng. Vì vậy phạm vi lan truyền thường chỉ giới hạn ở một bước và chi phí tính toán tương đối thấp.

AC-3 duy trì tính nhất quán cung giữa các biến. Khi một giá trị bị loại khỏi miền của một biến, sự thay đổi có thể tiếp tục được lan truyền sang các biến liên quan khác. Vì vậy AC-3 thu hẹp miền mạnh hơn và có thể phát hiện mâu thuẫn sớm hơn, nhưng chi phí tính toán cũng cao hơn Forward Checking.

b/ Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i/ Minimum Remaining Values (MRV)

MRV dùng để quyết định "nên gán biến nào trước"

Thuật toán chọn biến chưa được gán có **ít giá trị hợp lệ còn lại nhất**.

Ví dụ: $D(X) = \{1,2\}, D(Y) = \{1,2,3,4\}, D(Z) = \{2\}$

MRV sẽ chọn: Z vì Z chỉ còn 1 giá trị.

Ý tưởng là xử lý biến khó nhất trước. Nếu biến đó không còn giá trị hợp lệ, ta sẽ phát hiện thất bại ngay thay vì gán nhiều biến khác rồi mới phải backtrack.

=>MRV (Minimum Remaining Values) chọn biến chưa được gán có ít giá trị hợp lệ còn lại nhất. Heuristic này ưu tiên giải quyết biến bị ràng buộc nhiều nhất trước, nhờ đó có thể phát hiện dead end sớm và giảm số lượng nhánh phải tìm kiếm.

ii/ Least Constraining Value (LCV)

LCV dùng để quyết định "sau khi chọn biến, nên thử giá trị nào trước"

LCV chọn giá trị làm **loại ít giá trị nhất khỏi miền của các biến chưa được gán khác**.

Ví dụ ta đang gán biến X: $D(X) = \{1,2\}$

Nếu chọn X = 1 làm mất 4 khả năng của các biến khác, chọn X = 2 chỉ làm mất 1 khả năng thì LCV chọn: $X = 2$

Ý tưởng là giữ cho các biến còn lại càng nhiều lựa chọn càng tốt.

LCV (Least Constraining Value) chọn giá trị gây ít hạn chế nhất cho các biến chưa được gán. Giá trị được chọn sẽ loại ít giá trị nhất khỏi miền của các biến lân cận, từ đó giữ lại nhiều khả năng cho các bước tiếp theo và giảm khả năng phải backtrack.

MRV và LCV thường dùng cùng nhau vì: MRV chọn biến có ít giá trị hợp lệ còn lại nhất nhằm xử lý biến khó trước và phát hiện dead end sớm. LCV chọn giá trị gây ít ràng buộc nhất cho các biến khác nhằm giữ lại nhiều lựa chọn cho các bước tiếp theo. Khi kết hợp, MRV giúp giảm chiều rộng không gian tìm kiếm còn LCV giúp giảm khả năng backtracking.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a/ Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

1. Biến

$$X = \{A, B, C, D, E\}$$

Mỗi biến biểu diễn một máy bay cần được bố trí. Một giá trị gán cho máy bay có dạng: (t,r). Trong đó $t \in \{1,2,3,4\}$: khung giờ và $r \in \{QT, N\text{Đ}\}$: đường băng quốc tế hoặc nội địa

2. Miền giá trị ban đầu

Nếu chưa xét các ràng buộc riêng, mỗi máy bay có miền:

$$D(X_i) = \{1,2,3,4\} \times \{QT, N\text{Đ}\}$$

tức là $\{(1, QT), (2, QT), (3, QT), (4, QT), (1, N\text{Đ}), (2, N\text{Đ}), (3, N\text{Đ}), (4, N\text{Đ})\}$

3. Ràng buộc unary

Theo đề: B phải hạ cánh ở giờ 1, D chỉ có thể hạ cánh từ giờ 3 trở đi, A chỉ chờ tối đa đến giờ 2.

Máy bay B: $t_B = 1 \Rightarrow D(B) = \{(1, QT), (1, N\text{Đ})\}$

Máy bay D: $t_D \geq 3 \Rightarrow D(D) = \{(3, QT), (3, N\text{Đ}), (4, QT), (4, N\text{Đ})\}$

Máy bay A: $t_A \leq 2 \Rightarrow D(A) = \{(1, QT), (2, QT), (1, N\text{Đ}), (2, N\text{Đ})\}$

C và E chưa có ràng buộc unary về thời gian.

4. Ràng buộc binary

D phải trước C

Do hành khách nối chuyến từ D sang C: $t_D < t_C$

Không hai máy bay cùng dùng một đường băng ở cùng thời điểm

Với mọi cặp máy bay X_i, X_j:

$$(t_i, r_i) \neq (t_j, r_j)$$

Nghĩa là hai máy bay được phép cùng giờ **nếu ở hai đường băng khác nhau**, nhưng không được cùng giờ và cùng đường băng.

b/ Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

Vì đường băng đã được xác định sẵn nên miền chỉ cần chứa **khung giờ**.

Miền mới trước khi áp dụng unary

$$D(A) = D(B) = D(C) = D(D) = D(E) = \{1,2,3,4\}$$

Sau đó ta vẫn còn các unary: $A \leq 2, B = 1, D \geq 3$

Đồ thị ràng buộc

Do A, B, C dùng cùng đường băng quốc tế nên phải khác giờ nhau: $A \neq B, A \neq C, B \neq C$

D và E cùng dùng đường băng nội địa: $D \neq E$

Ngoài ra: $D < C$

Do đó có các cạnh: $A - B, A - C, B - C, C - D, D - E$

Trong đó:

- $A - B, B - C, A - C, D - E$: ràng buộc khác giờ;
- $C - D$: $D < C$

c/ Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Sau khi áp dụng unary constraints:

$$A = \{1,2\}, B = \{1\}, C = \{1,2,3,4\}, D = \{3,4\}, E = \{1,2,3,4\}$$

Do $B = 1$ và $A \neq B$, loại 1 khỏi miền của A, nên $A = \{2\}$. Đồng thời $B \neq C$ làm loại 1 khỏi C. Vì $A = 2$ và $A \neq C$, tiếp tục loại 2 khỏi C, nên $C = \{3,4\}$.

Với ràng buộc $D < C$, giá trị $D = 4$ không có giá trị C nào hỗ trợ nên bị loại; do đó $D = 3$. Khi D bằng 3, giá trị $C = 3$ cũng không còn hỗ trợ nên bị loại, suy ra $C = 4$.

Cuối cùng do $D \neq E$ và $D = 3$, loại 3 khỏi E:

$$A = \{2\}, B = \{1\}, C = \{4\}, D = \{3\}, E = \{1,2,4\}$$

d/ Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

Sau khi áp dụng unary constraints:

$$A = \{1,2\}, B = \{1\}, C = \{1,2,3,4\}, D = \{3,4\}, E = \{1,2,3,4\}$$

Theo MRV, B được chọn trước và gán $B = 1$. Forward Checking loại 1 khỏi A và C, nên $A = 2$, $C = \{2,3,4\}$. Tiếp theo chọn $A = 2$, làm miền C còn $\{3,4\}$.

C và D đều có hai giá trị nên theo quy tắc phá hòa chọn C trước. Thử giá trị nhỏ nhất $C = 3$. Ràng buộc $D < C$ yêu cầu $D < 3$, nhưng miền D là $\{3,4\}$, vì vậy miền D trở thành rỗng và xảy ra dead end. Thuật toán backtrack khỏi $C = 3$.

Tiếp theo thử $C = 4$. Khi đó $D < C$ làm miền D còn $\{3\}$, nên gán $D = 3$. Forward Checking loại 3 khỏi E, do đó $E = \{1,2,4\}$. Chọn giá trị nhỏ nhất $E = 1$.

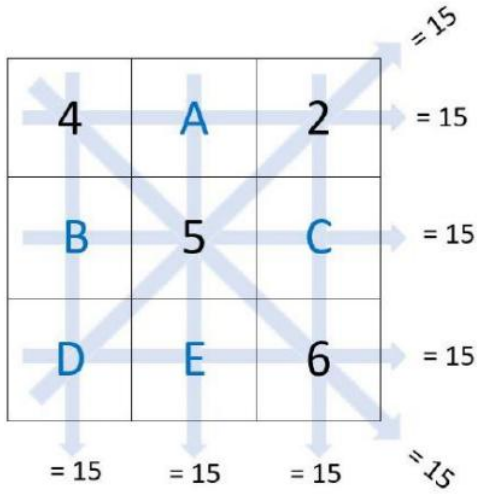
Thứ tự các phép gán là:

$$(B,1),(A,2),(C,3) \rightarrow Backtrack,(C,4),(D,3),(E,1)$$

Nghiệm cuối cùng: $A = 2, B = 1, C = 4, D = 3, E = 1$

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a/ Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Các biến của CSP là A, B, C, D, E, với miền ban đầu:

$$D(A) = D(B) = D(C) = D(D) = D(E) = \{1,3,7,8,9\}$$

và các biến phải thỏa ràng buộc $AllDiff(A, B, C, D, E)$.

Các ràng buộc tổng là:

$$A = 9, B + C = 10, D + E = 9, B + D = 11, A + E = 10, C = 7, D = 8$$

Mỗi hàng, cột và hai đường chéo đều phải có tổng bằng 15.

b/ Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

A — A+E=10 — E — D+E=9 — D — B+D=11 — B — B+C=10 — C

Các điều kiện: $A = 9, C = 7, D = 8$ là ràng buộc unary nên không tạo cạnh riêng trong đồ thị.

c/ Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Sau khi áp dụng các ràng buộc unary, ta có: $A = \{9\}, C = \{7\}, D = \{8\}$

Với ràng buộc $A + E = 10$, do $A = 9$ nên $E = 1$

Tiếp theo, từ $B + D = 11$, do $D = 8$ nên $B = 3$

Các ràng buộc còn lại được kiểm tra: $B + C = 3 + 7 = 10, D + E = 8 + 1 = 9$

Vì vậy sau AC-3, miền của các biến là:

$$A = \{9\}, B = \{3\}, C = \{7\}, D = \{8\}, E = \{1\}$$

Nghiệm Magic Square thu được là:

4 9 2

3 5 7

8 1 6

Mỗi hàng, mỗi cột và hai đường chéo đều có tổng bằng 15.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá}
- Sunny = {Đỏ, Vàng, Hồng}
- Ellyse = {Đỏ, Hồng}

a/ Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán

Bước 1: Chọn biến bằng MRV

MRV chọn biến có **ít giá trị còn lại nhất**.

Số lượng giá trị: AJ: 3, Sunny: 3, Ellyse: 2

Do đó chọn Ellyse

Bước 2: Chọn giá trị cho Ellyse bằng LCV

Ellyse có: {Đỏ, Hồng}. Xét mức độ ràng buộc của từng màu.

Nếu chọn Ellyse = Đỏ, màu Đỏ sẽ bị loại khỏi AJ và Sunny → làm mất tổng cộng **2 lựa chọn**.

Nếu chọn Ellyse = Hồng, màu Hồng chỉ bị loại khỏi Sunny → chỉ làm mất **1 lựa chọn**.

LCV chọn giá trị ít ràng buộc nhất nên: $Ellyse = Hồng$

Sau Forward Checking: $Sunny = \{Đỏ, Vàng\}; AJ = \{Đỏ, Vàng, Xanh\ lá\}$

Bước 3: MRV chọn Sunny

Hiện tại Sunny còn 2 giá trị, AJ còn 3 giá trị nên chọn Sunny

Sunny có: {Đỏ, Vàng}

Nếu chọn Đỏ thì loại Đỏ khỏi AJ.

Nếu chọn Vàng thì loại Vàng khỏi AJ.

Cả hai đều làm mất 1 giá trị nên hòa. Theo quy tắc thứ tự cầu vồng, Đỏ đứng trước Vàng, nên chọn $Sunny = Đỏ$

Khi đó:$AJ = \{Vàng, Xanh\ lá\}$

Bước 4: Gán AJ

AJ là biến cuối cùng.

Hai giá trị Vàng và Xanh lá đều hợp lệ. Theo thứ tự cầu vồng chọn Vàng trước: $AJ = Vàng$

Ta được nghiệm: $Ellyse = Hồng, Sunny = Đỏ, AJ = Vàng$

Không xảy ra dead end nên số lần backtrack = 0.

Thứ tự gán: (Ellyse, Hồng) → (Sunny, Đỏ) → (AJ, Vàng)

b/ Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Khi AJ và Sunny hòa nhau về số giá trị còn lại, giả sử chọn AJ trước theo thứ tự xuất hiện trong đề.

Bước 1: chọn AJ

Dùng Most Constraining Value: **AJ = Đỏ**

Đỏ xuất hiện trong miền của: Sunny, Ellyse → loại tổng cộng 2 lựa chọn.

AJ = Vàng: Vàng chỉ xuất hiện ở Sunny → loại 1 lựa chọn.

AJ = Xanh lá: Không xuất hiện ở miền của Sunny hoặc Ellyse → không loại lựa chọn nào.

Most Constraining Value chọn giá trị gây ràng buộc nhiều nhất: AJ = Đỏ

Sau đó: Sunny = {Vàng, Hồng}, Ellyse = {Hồng}

Bước 2: Maximum Remaining Values chọn Sunny

Sunny có 2 giá trị, Ellyse chỉ có 1 nên chọn: Sunny

Sunny có: {Vàng, Hồng}

Sunny = Vàng: Không ảnh hưởng đến Ellyse vì Ellyse không có Vàng → loại 0 giá trị.

Sunny = Hồng: Loại Hồng khỏi Ellyse.

Nhưng Ellyse chỉ có: {Hồng} nên miền Ellyse trở thành: {}

Most Constraining Value cố tình chọn giá trị gây ràng buộc mạnh nhất nên chọn: Sunny = Hồng

Ngay lập tức xảy ra: D(Ellyse) = Nothing → dead end, thuật toán phải backtrack.

Bước 3: Backtrack Sunny

Hoàn tác: Sunny = Hồng và thử giá trị còn lại: Sunny = Vàng

Khi đó Ellyse vẫn còn Hồng

Bước 4: Gán Ellyse

Ellyse = Hồng

Nghiệm: AJ = Đỏ, Sunny = Vàng, Ellyse = Hồng

Số lần backtrack: 1

Thứ tự gán: (AJ, Đỏ) → (Sunny, Hồng) → Backtrack → (Sunny, Vàng) → (Ellyse, Hồng)